



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 11.b: Extensión de Formularios Reactivos (FormArray y Validadores) y Colecciones en H2

Resumen

Esta práctica guiada es la continuación directa de la **Práctica 11.a**. Ahora que nuestro sistema básico de **TechConf** está funcionando con una base de datos H2 en memoria y un formulario reactivo básico en Angular, escalaremos su complejidad para enfrentar escenarios empresariales más reales.

En el **Back-End**, modificaremos la base de datos para soportar colecciones de elementos básicos (arreglos de *Strings*) usando `@ElementCollection`, permitiendo que una charla tenga múltiples "etiquetas" (tags) de búsqueda, y actualizaremos nuestro script semilla `data.sql`. En el **Front-End**, llevaremos el poder de los Formularios Reactivos al siguiente nivel: implementaremos un **FormArray** para permitir la captura dinámica de múltiples etiquetas (agregando y removiendo campos de texto al vuelo) y crearemos un **Validador Personalizado Cruzado (Cross-Field**

Validator) para garantizar coherencia en la lógica de negocio de las fechas.

Metadatos del Laboratorio

- **Tiempo estimado:** 2.5 horas.
- **Requisitos:** Haber completado funcionalmente la Práctica Guiada 11.a.

Metas de Aprendizaje:

1. Mapear colecciones de tipos básicos en Spring Data JPA usando `@ElementCollection` y `@CollectionTable`.
2. Actualizar scripts ANSI SQL (`data.sql`) para insertar datos en tablas relacionales secundarias de forma transparente.
3. Implementar `FormArray` en Angular para agregar y eliminar controles de formulario dinámicamente desde la Interfaz Gráfica.
4. Diseñar y aplicar un Validador Personalizado a nivel de `FormGroup` (Cross-Field Validation).

Parte 1: Ampliación del Back-End (H2 y JPA Colecciones)

Paso 1.1: Modificación de la Entidad `Charla`

Queremos que cada `charla` pertenezca a un rango de fechas y tenga múltiples etiquetas dinámicas (ej. "Java", "Backend", "AI"). Abra `charla.java` y agregue estos nuevos atributos junto con las anotaciones JPA para manejar colecciones de elementos:

```
// Agregue estas importaciones
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

// Dentro de la clase Charla, agregue:
```

```

@Column(name = "fecha_inicio")
private LocalDate fechaInicio;

@Column(name = "fecha_fin")
private LocalDate fechaFin;

// Colección embebida para almacenar múltiples etiquetas (tags)
@ElementCollection
@CollectionTable(name = "charla_etiquetas", joinColumns = @JoinColumn(name = "charla_id"))
@Column(name = "etiqueta")
private List<String> etiquetas = new ArrayList<>();

// IMPORTANTE: Genere los Getters y Setters para fechaInicio, fechaFin y etiquetas.

```

Paso 1.2: Actualización de `data.sql`

Dado que añadimos nuevas columnas y una tabla de colección (`charla_etiquetas`), debemos actualizar nuestro script de inicialización. Modifique `src/main/resources/data.sql`:

```

-- Inserción de Charlas principales con fechas
INSERT INTO charla (titulo, expositor, nivel, email_contacto, fecha_inicio, fecha_fin)
VALUES ('Introduccion a la Inteligencia Artificial', 'Dra. Maria Rojas', 'Principiante', 'maria@ai-

INSERT INTO charla (titulo, expositor, nivel, email_contacto, fecha_inicio, fecha_fin)
VALUES ('Microservicios con Spring Cloud', 'Ing. Carlos Brenes', 'Avanzado', 'carlos@spring.io', '2

INSERT INTO charla (titulo, expositor, nivel, email_contacto, fecha_inicio, fecha_fin)
VALUES ('Angular 18: Señales y Standalone', 'Licda. Laura Gomez', 'Intermedio', 'laura@angular.dev'

-- Inserción de Etiquetas (Tags) en la tabla relacional generada por @ElementCollection
-- El charla_id corresponde al orden de inserción (1, 2, 3) al ser IDENTITY
INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (1, 'IA');
INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (1, 'Machine Learning');

INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (2, 'Spring Boot');
INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (2, 'Backend');
INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (2, 'Nube');

INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (3, 'Angular');
INSERT INTO charla_etiquetas (charla_id, etiqueta) VALUES (3, 'Frontend');

```

Reinicie su aplicación Spring Boot y verifique en la consola de H2 (/h2-console) que se ha creado la tabla `CHARLA_ETIQUETAS` poblada correctamente.

Parte 2: Formularios Reactivos Avanzados (Angular)

Paso 2.1: Actualizar la Interfaz y el Servicio

En `charla.service.ts` (Angular), actualice la interfaz `Charla` para soportar las nuevas propiedades:

```
export interface Charla {
  id?: number;
  titulo: string;
  expositor: string;
  nivel: string;
  emailContacto: string;
  fechaInicio: string; // Formato YYYY-MM-DD
  fechaFin: string;    // Formato YYYY-MM-DD
  etiquetas: string[]; // Arreglo dinámico de strings
}
```

Paso 2.2: Validador Personalizado (Cross-Field Validator)

Necesitamos asegurar lógicamente que la `fechaFin` **nunca** sea anterior a la `fechaInicio`. Esta es una validación cruzada porque depende de dos campos distintos.

En `charla-registro.component.ts`, **fuera** de la clase del componente (o en un archivo separado), declare el siguiente validador:

```
import { AbstractControl, ValidationErrors, ValidatorFn } from '@angular/forms';

// Validador cruzado para asegurar coherencia de fechas
export const validarRangoFechas: ValidatorFn = (control: AbstractControl): ValidationErrors | null {
  const inicio = control.get('fechaInicio')?.value;
  const fin = control.get('fechaFin')?.value;

  if (inicio && fin) {
    const dInicio = new Date(inicio);
```

```

    const dFin = new Date(fin);
    // Si la fecha final es menor a la de inicio, retornamos un objeto de error
    if (dFin < dInicio) {
        return { fechasInvalidas: true };
    }
}
return null; // Nulo significa que la validación pasó con éxito
};

```

Paso 2.3: Implementar FormArray y el Validador en el FormGroup

Dentro de `CharlaRegistroComponent`, modifique la estructura del `registroForm`. Para construir un arreglo dinámico, utilizaremos `FormBuilder` (es más limpio que hacer `new FormGroup` recursivamente).

```

// Asegúrese de importar FormBuilder, FormArray y el validador cruzado en la cabecera
import { FormBuilder, FormArray } from '@angular/forms';

export class CharlaRegistroComponent implements OnInit {
    private fb = inject(FormBuilder);

    // ... (código previo) ...

    // 1. Redefinir el formulario con FormBuilder, añadiendo fechas, el validador cruzado y el FormArray
    registroForm = this.fb.group({
        titulo: ['', [Validators.required, Validators.minLength(5)]],
        expositor: ['', [Validators.required]],
        nivel: ['Principiante', [Validators.required]],
        emailContacto: ['', [Validators.required, Validators.email]],
        fechaInicio: ['', [Validators.required]],
        fechaFin: ['', [Validators.required]],
        etiquetas: this.fb.array([
            this.fb.control('', Validators.required) // Iniciamos con un input vacío obligatorio
        ])
    }, { validators: validarRangoFechas }); // Aplicamos el validador a todo el grupo!

    // 2. Getter para interactuar fácilmente con el FormArray
    get etiquetasArray() {
        return this.registroForm.get('etiquetas') as FormArray;
    }

    // 3. Métodos para mutar el FormArray dinámicamente
    agregarEtiqueta() {
        this.etiquetasArray.push(this.fb.control('', Validators.required));
    }
}

```

```

    }
    removeEtiqueta(index: number) {
      // Evitar borrar si solo queda uno
      if (this.etiquetasArray.length > 1) {
        this.etiquetasArray.removeAt(index);
      }
    }
  }

  // ... (modificar onSubmit para soportar reinicio de FormArray si lo desea)
}

```

Paso 2.4: Renderizado Dinámico en la Vista (HTML)

Actualice `charla-registro.component.html` para incluir los inputs de fechas, mostrar los errores cruzados e iterar el arreglo de etiquetas dinámicamente:

```

<!-- Agregue esto dentro del formulario antes del botón de Submit -->

<div class="fechas-container">
  <div class="form-group">
    <label>Fecha de Inicio:</label>
    <input type="date" formControlName="fechaInicio">
  </div>

  <div class="form-group">
    <label>Fecha de Fin:</label>
    <input type="date" formControlName="fechaFin">
  </div>
</div>

<!-- Renderizar error de validación cruzada del grupo entero -->
@if(registroForm.errors?.['fechasInvalidas'] && registroForm.touched) {
  <div class="error general-error">La fecha de fin no puede ser anterior a la de inicio.</div>
}

<fieldset formArrayName="etiquetas" class="tags-section">
  <legend>Etiquetas Tecnológicas (Tags)</legend>

  <!-- Iterar sobre el FormArray usando la sintaxis de Angular Control Flow -->
  @for(control of etiquetasArray.controls; track $index) {
    <div class="dynamic-row">
      <input type="text" [formControlName]="$index" placeholder="Ej. Spring Boot">
    </div>
  }
</fieldset>

```

```

    @if(etiquetasArray.length > 1) {
        <button type="button" class="btn-danger" (click)="removerEtiqueta($index)">X</button>
    }

    @if(control.invalid && control.touched) {
        <span class="error inline-error">*</span>
    }
</div>
}

<button type="button" class="btn-secondary" (click)="agregarEtiqueta()">+ Añadir Otra Etiqueta</button>
</fieldset>

```

3. Reto Evaluativo Autónomo (Finalización de Práctica)

Compile y ejecute ambas aplicaciones. Para dar por terminada exitosamente la práctica, demuestre al docente o tutor lo siguiente: 1. Que al ingresar una fecha de fin anterior a la de inicio, aparece el mensaje *"La fecha de fin no puede ser anterior a la de inicio"* y **el botón de "Guardar Charla" se deshabilita automáticamente**. 2. Que puede presionar "+ Añadir Otra Etiqueta" para generar 3 campos de etiqueta diferentes e ingresar valores en ellos (ej: "Docker", "DevOps", "CI/CD"). 3. Que al guardar exitosamente la charla, los datos y las múltiples etiquetas viajan a Spring Boot, se persisten en H2 (puede comprobarlo en /h2-console en la tabla CHARLA_ETIQUETAS), y la tarjeta recién creada en el UI muestra los arreglos de etiquetas obtenidos de la API. (Tendrá que adaptar el `@for` de la lista en HTML para mostrar las etiquetas).

¡Excelente trabajo dominando la escalabilidad de formularios reactivos!